

The Chinese University of Hong Kong

ENGG1910: Demystifying Artificial Intelligence

Lab 2: Neural Translation Machine: Mandarin to Cantonese Guideline

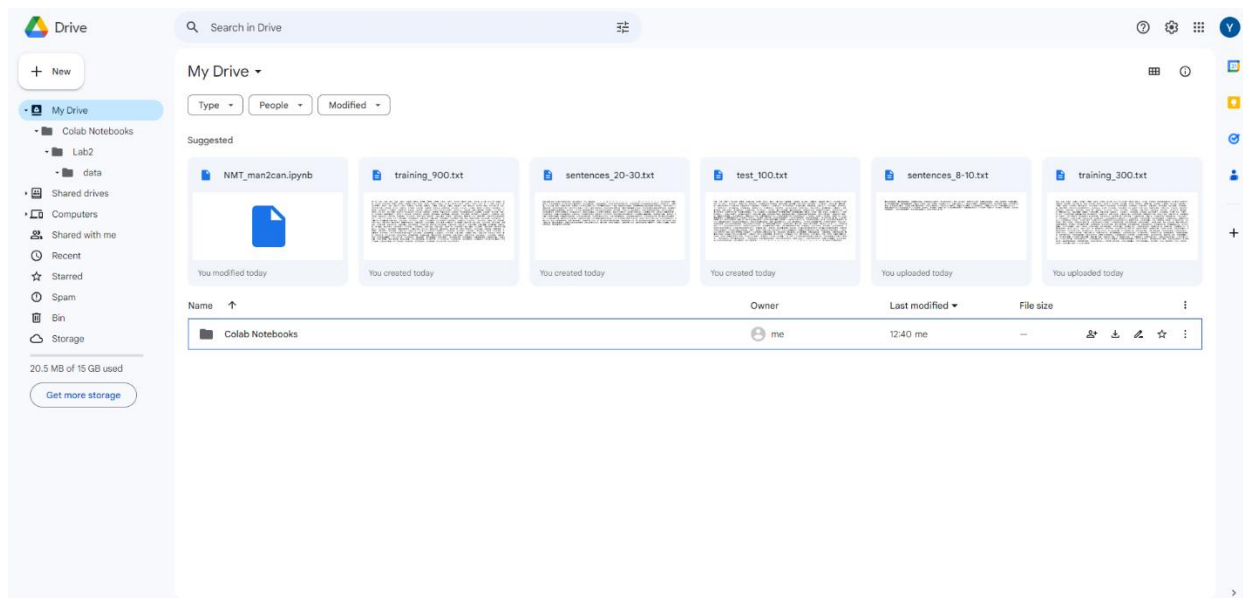
1. Objectives

- 1.1. In this lab, we will learn how to build a **Neural Machine Translation** (NMT) model for use in a practical scenario, i.e., translation from Mandarin to Cantonese.
- 1.2. The concepts of **pre-trained models** and **fine-tuning** will be introduced. You will learn how to fine-tune a pre-trained model with custom datasets via Google Drive and Google Colaboratory (Colab).
- 1.3. We are expected to investigate how model parameters, like sentence length and size of data, could affect an AI model's performance.

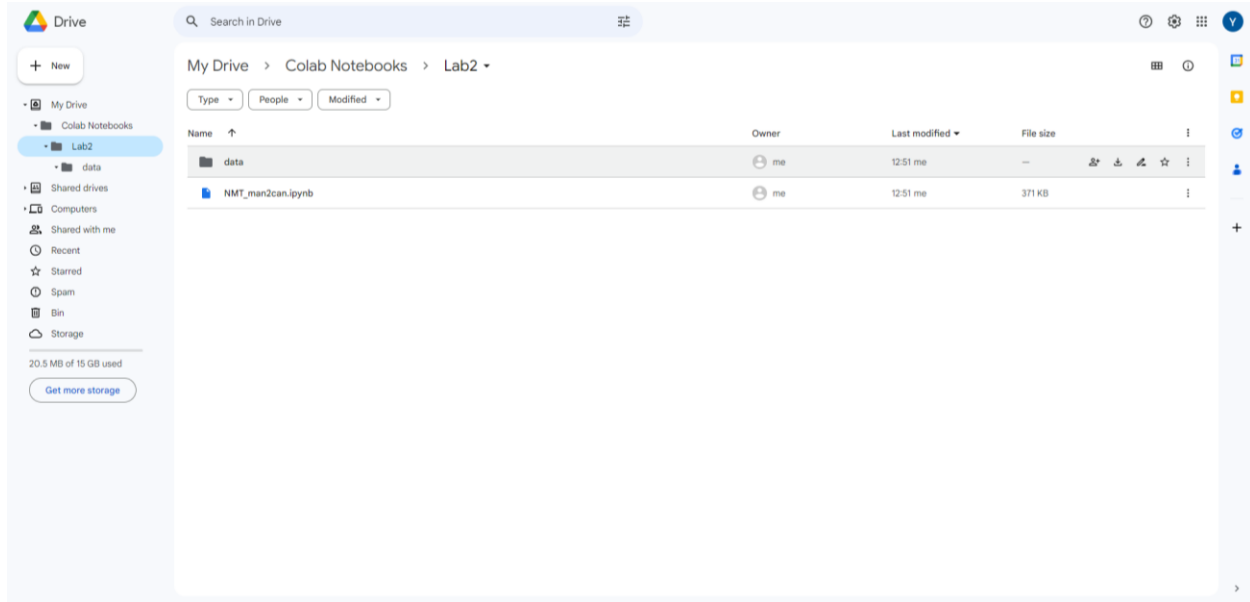
2. Detailed Steps:

2.1. Prepare the workplace on Google Colab.

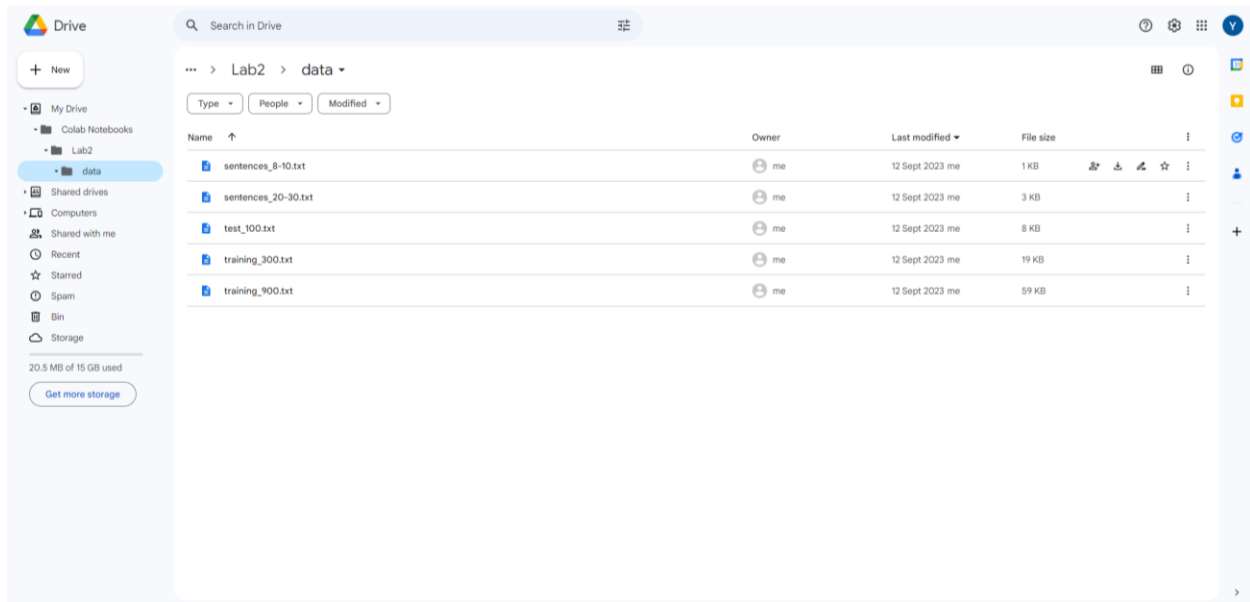
- 2.1.1. Log in to Google Drive using your Google account. Unzip the lab file and upload the “Colab Notebooks” folder to the **root directory** of your Google Drive.



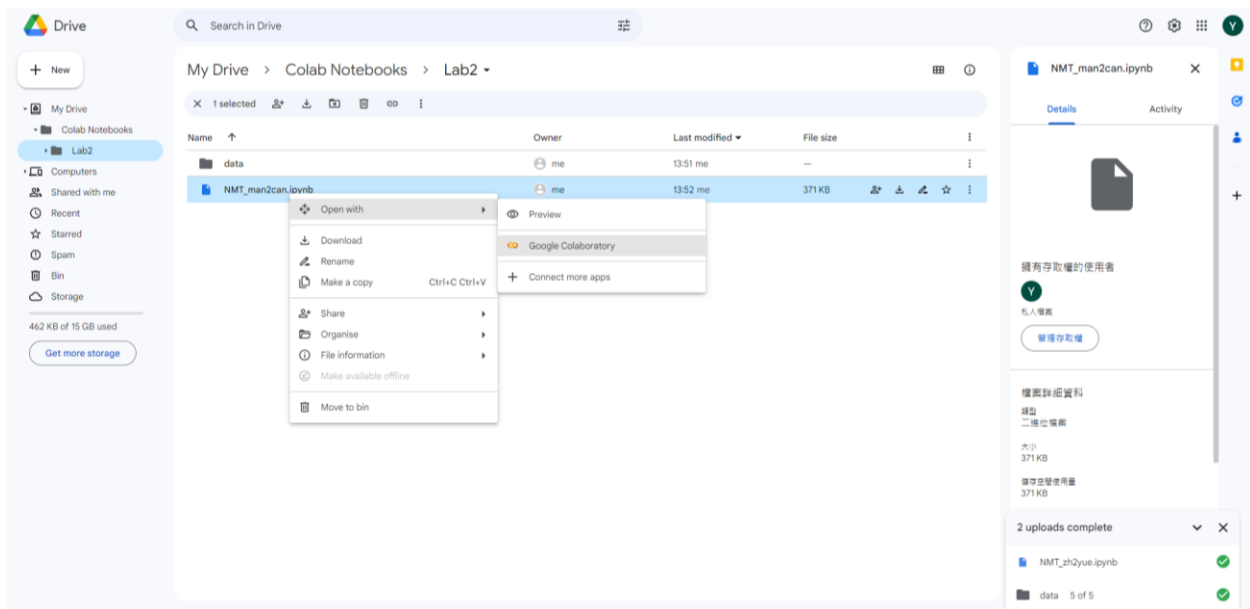
2.1.2. Open the “Colab Notebooks” folder and open the “Lab2” folder inside it. You should see a folder named “data” and a file named “NMT_man2can.ipynb”.



Inside the “data” folder, there should be five files, namely “sentences_8-10.txt”, “sentences_20-30.txt”, “test_100.txt”, “training_300.txt”, and “training_900.txt”.

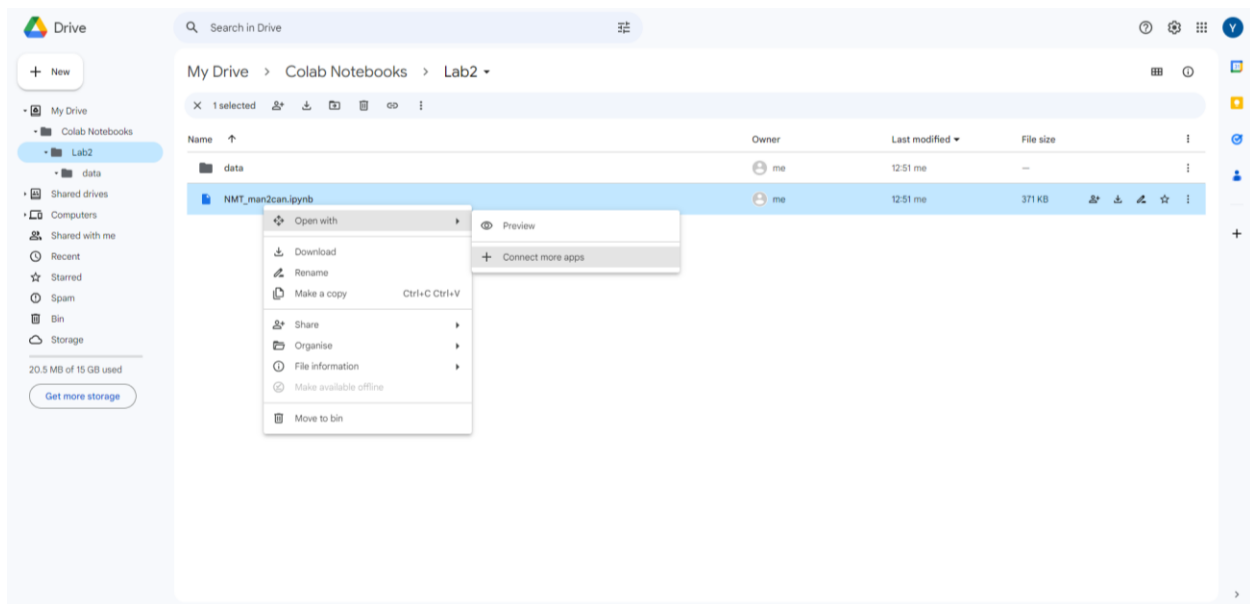


After checking that all text files are correct, return to the “Lab2” folder.

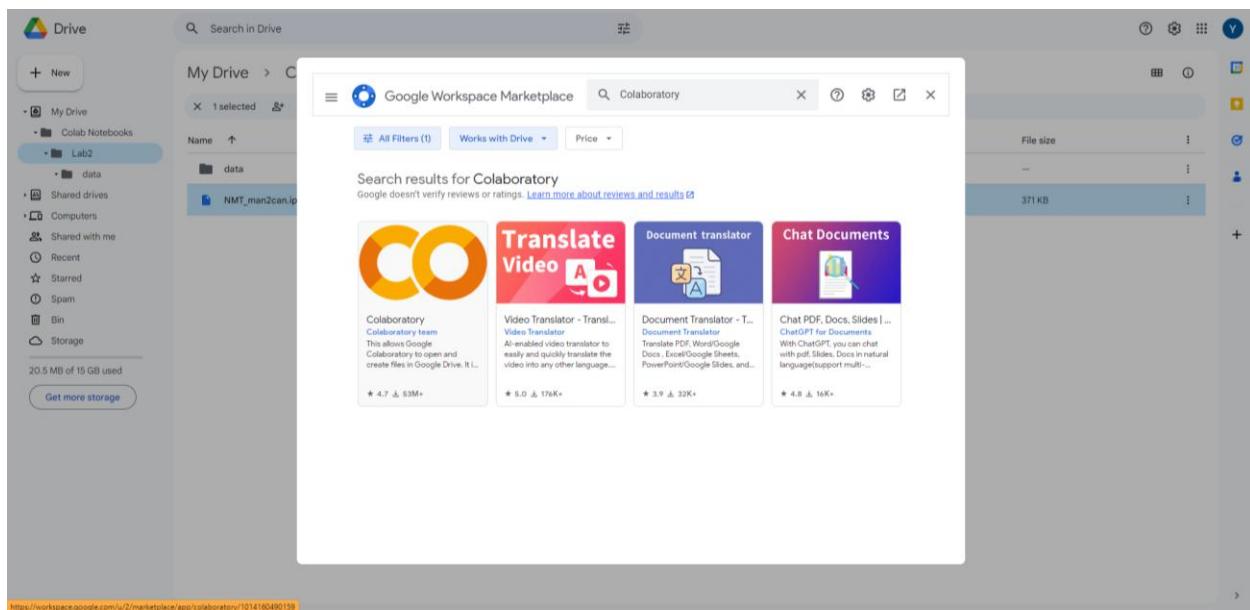
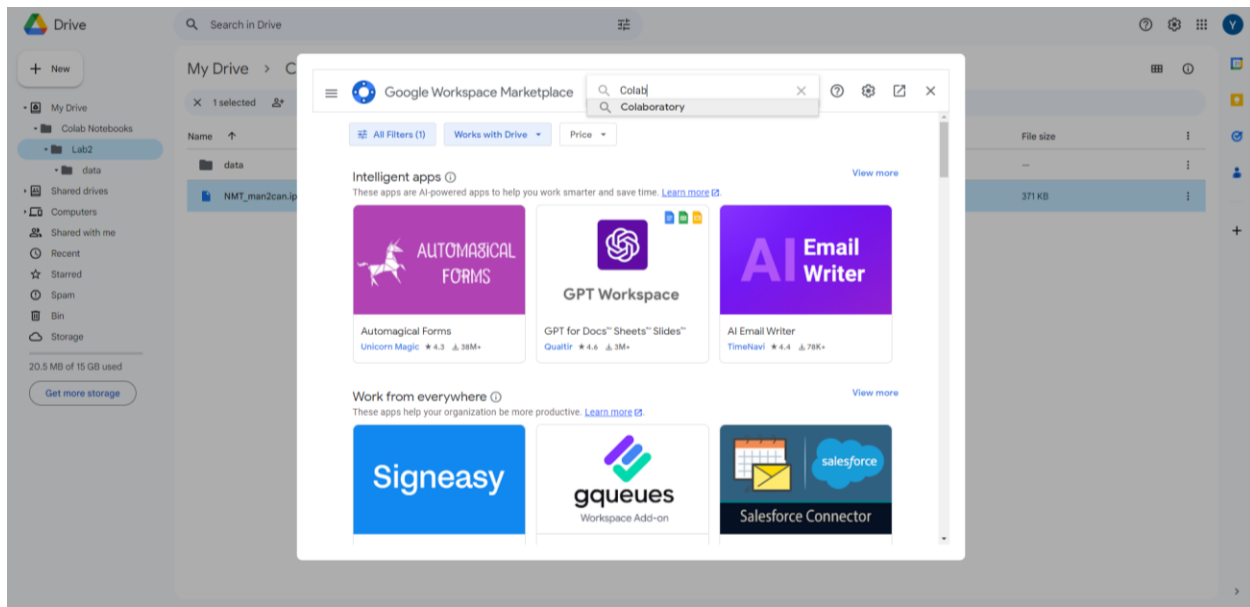


Right-click on the “NMT_man2can.ipynb” file and left-click to select “Connect more apps”.

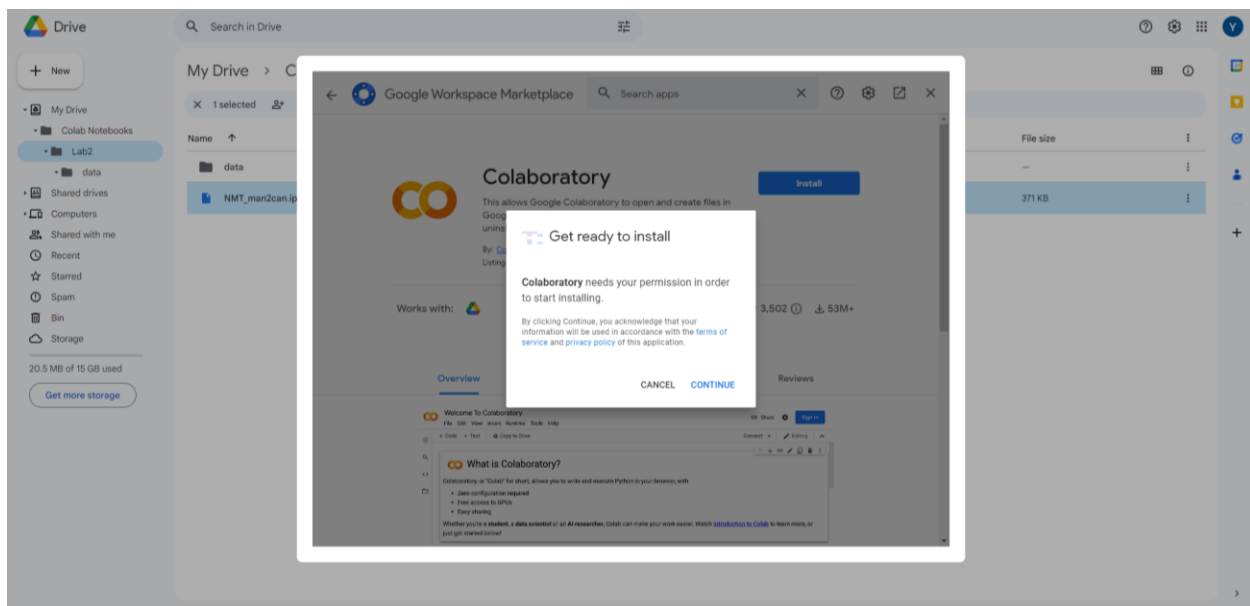
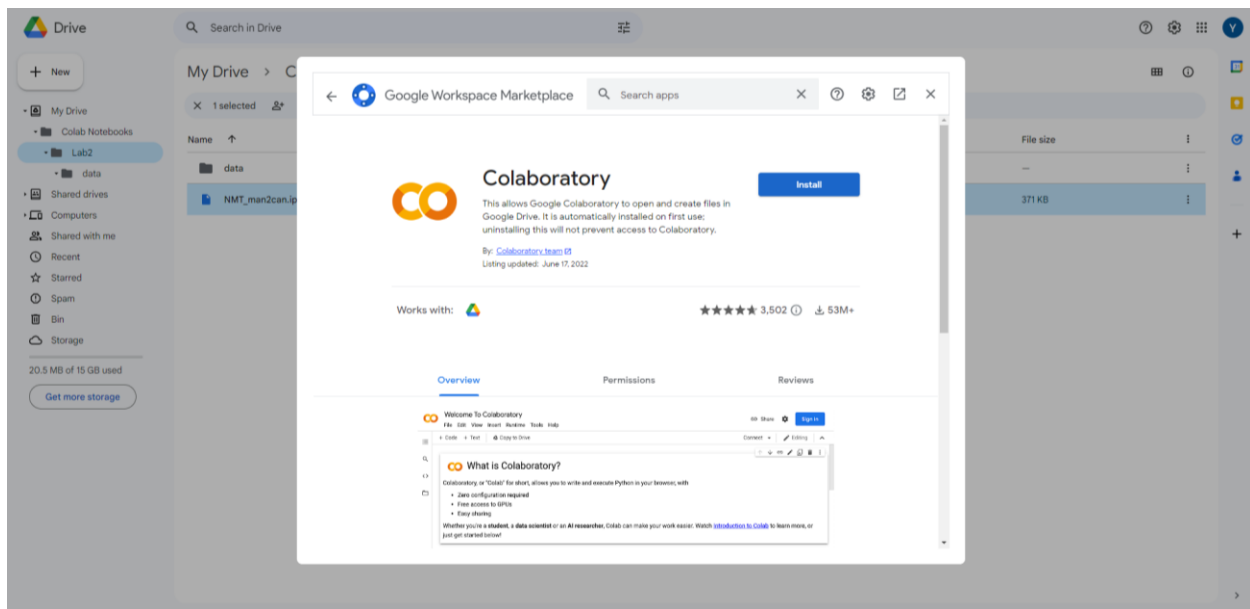
If you have already seen that option, you can select “Google Colaboratory” directly and proceed to Step 2.1.3.



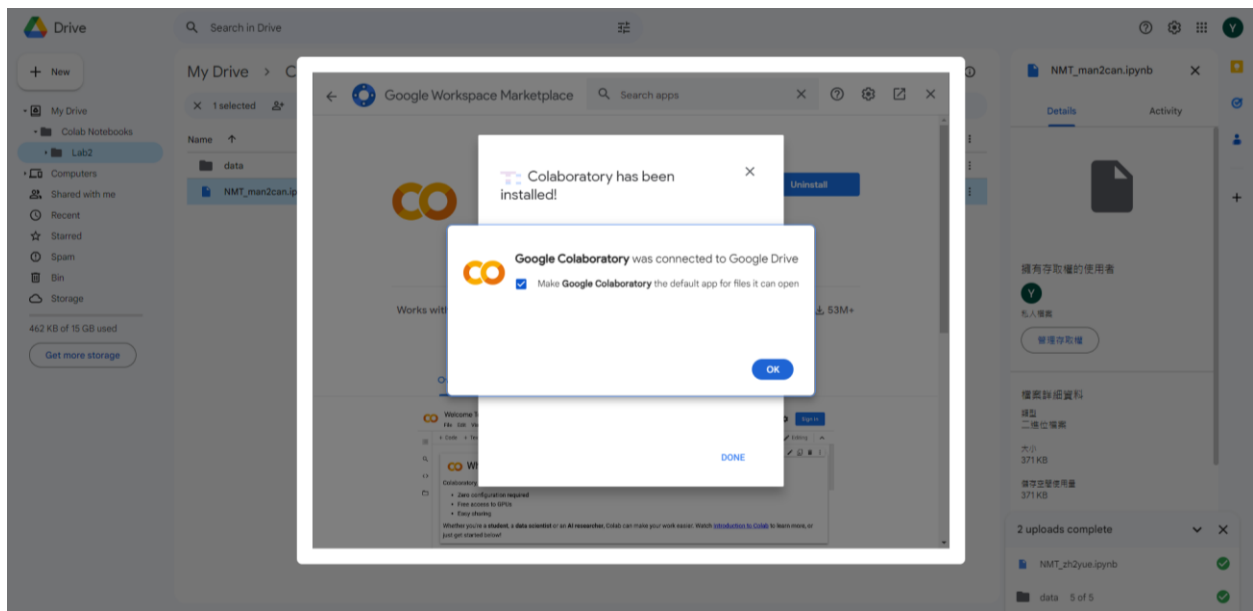
After clicking “Connect more apps”, type “Colaboratory” in the search box to search for the tool, or click on it in the prompt below the search box.



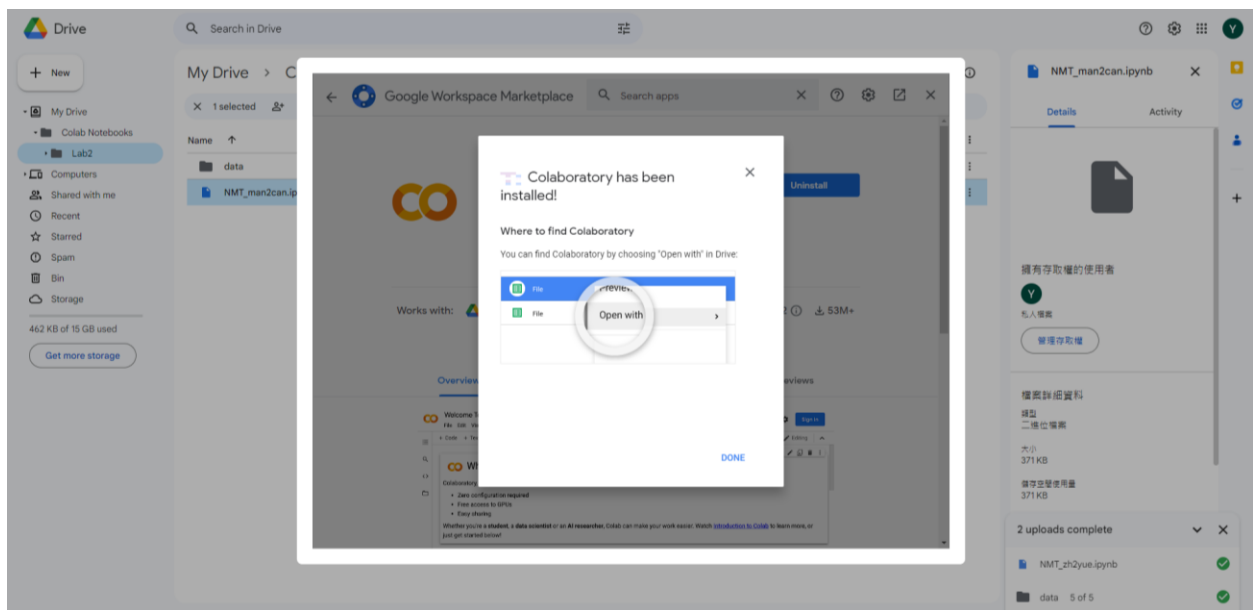
Click on the first option in the search results, then install Google Colaboratory.

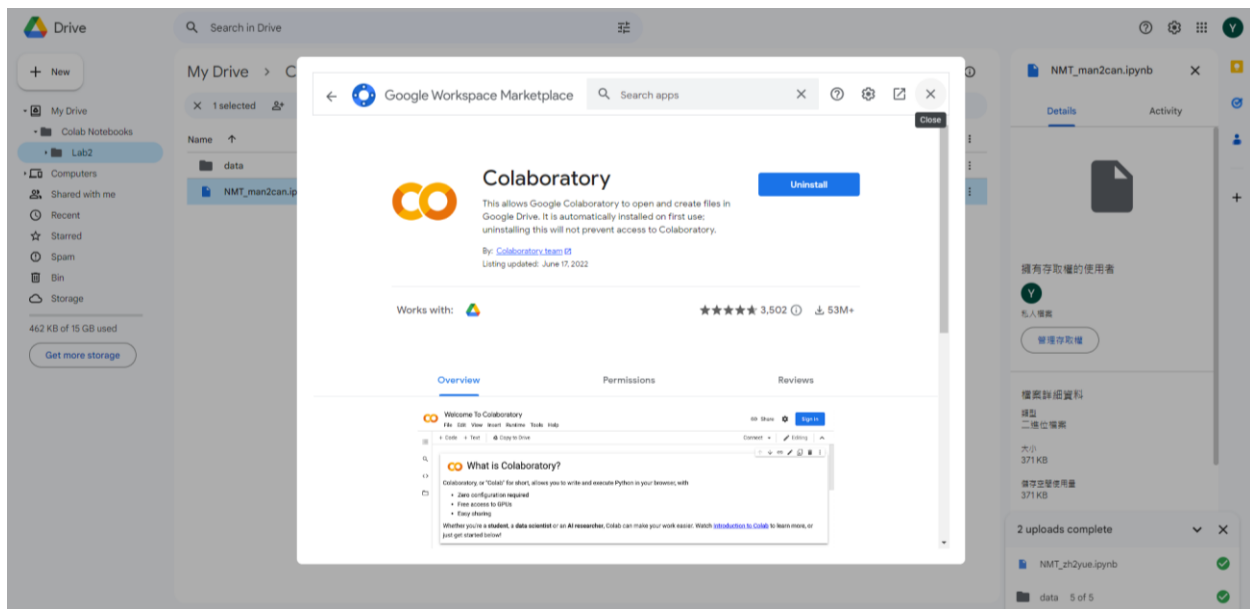


You may need to log in to your Google account again and give permission to install the tool.

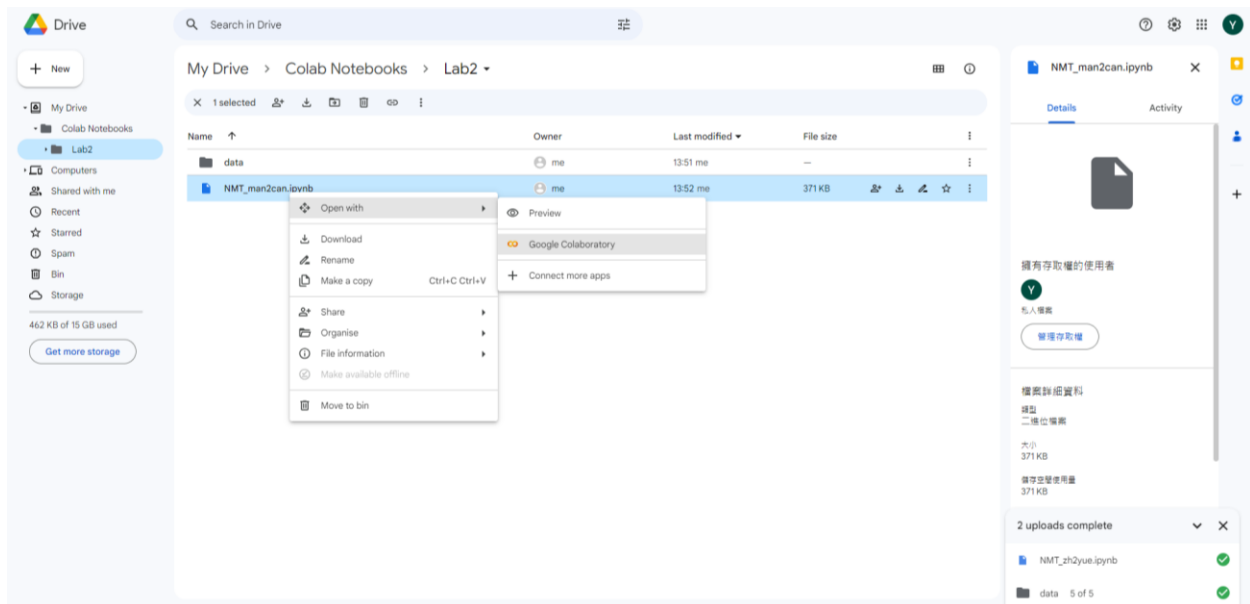


After installing Colaboratory, you can make it the default app for opening “ipynb” files if you wish.

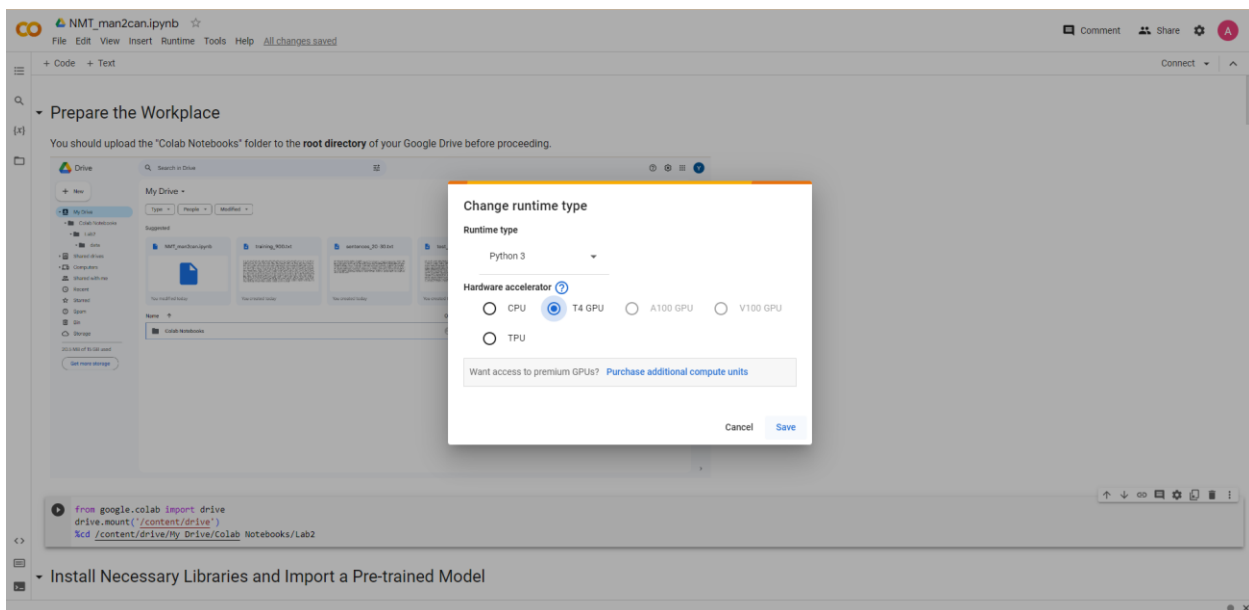
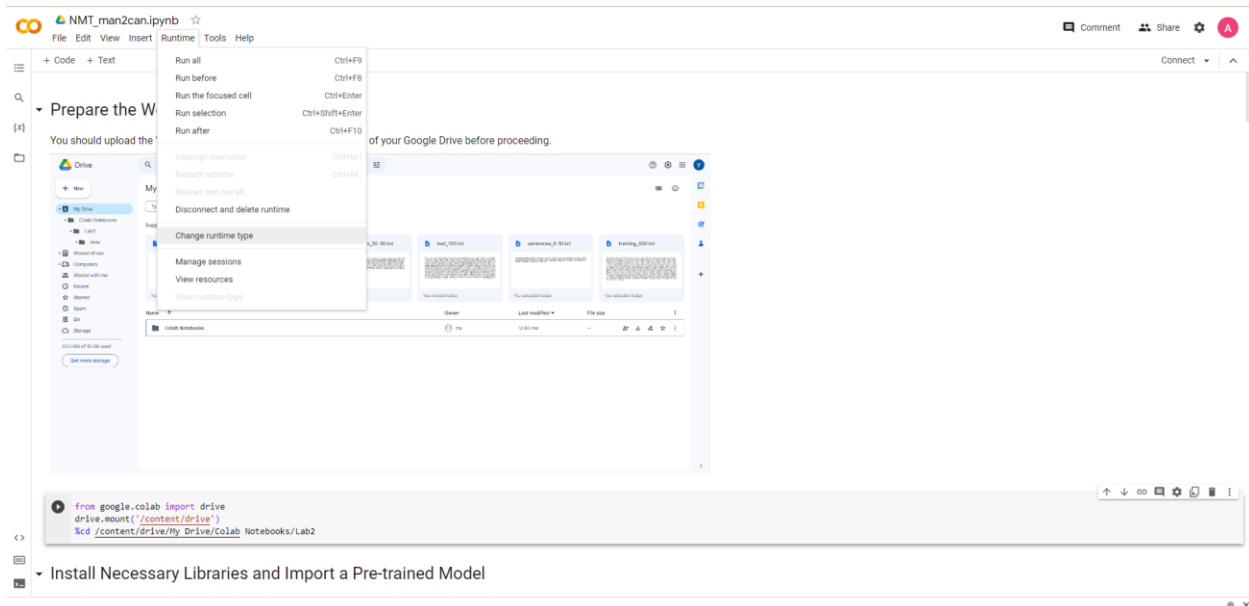




Close the windows and go back to the “Lab2” folder. Right-click the “NMT_man2can.ipynb” file again and select “Google Colaboratory” to start the lab.

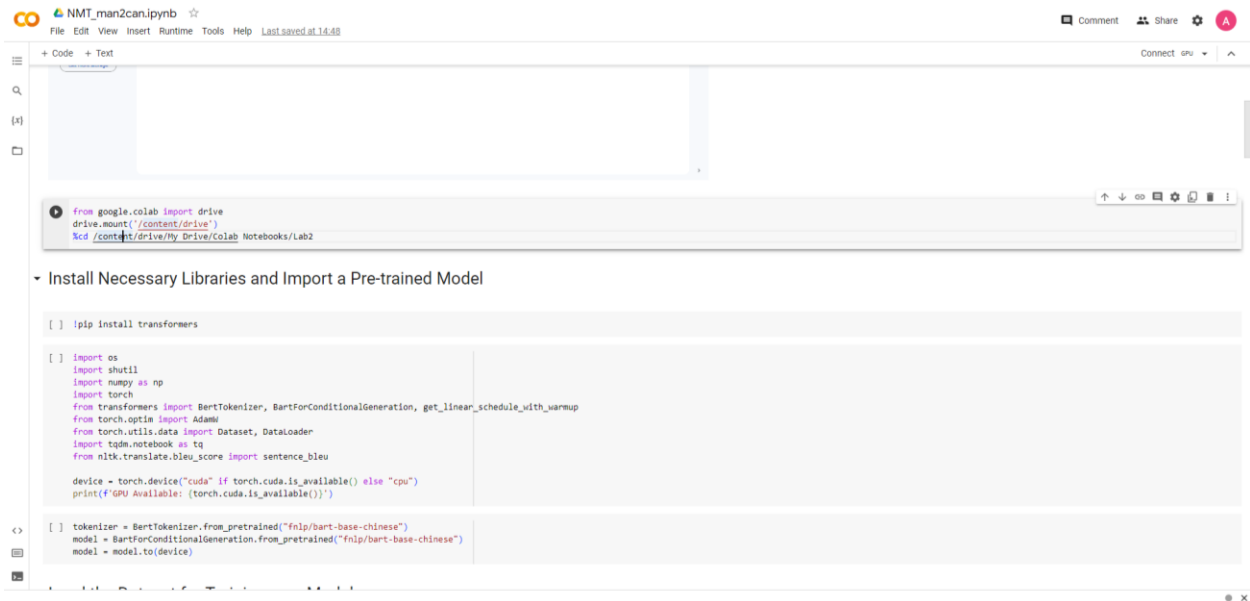


2.1.3. In the Colab interface, click “Runtime” at the top-left menu bar and select “Change runtime type” in the prompted menu.



Make sure “Python 3” is the Runtime Type, select “T4 GPU” as the Hardware accelerator, and save.

2.1.4. Go to the first runnable cell and click on the Run button, or select the cell and press “Ctrl” + “Enter” to run the cell.



```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/My Drive/Colab Notebooks/Lab2
```

Install Necessary Libraries and Import a Pre-trained Model

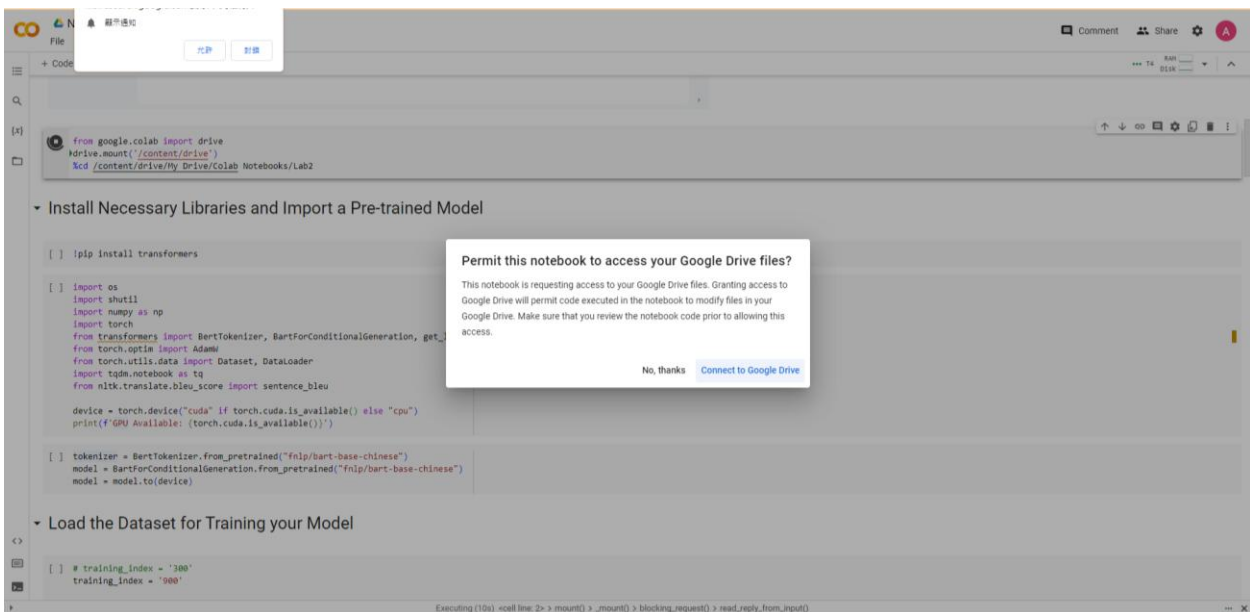
```
[ ] !pip install transformers
```

```
[ ] import os
import shutil
import numpy as np
import torch
from transformers import BertTokenizer, BartForConditionalGeneration, get_linear_schedule_with_warmup
from torch.optim import AdamW
from torch.utils.data import Dataset, DataLoader
import tqdm.notebook as tq
from nltk.translate.bleu_score import sentence_bleu

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f'GPU Available: {torch.cuda.is_available()}')
```

```
[ ] tokenizer = BertTokenizer.from_pretrained("fnlp/bart-base-chinese")
model = BartForConditionalGeneration.from_pretrained("fnlp/bart-base-chinese")
model = model.to(device)
```

New pop-up windows will be prompted to request your permission to access your Google Drive. Allow it to access and go through the login process again if required.



```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/My Drive/Colab Notebooks/Lab2
```

Install Necessary Libraries and Import a Pre-trained Model

```
[ ] !pip install transformers
```

```
[ ] import os
import shutil
import numpy as np
import torch
from transformers import BertTokenizer, BartForConditionalGeneration, get_linear_schedule_with_warmup
from torch.optim import AdamW
from torch.utils.data import Dataset, DataLoader
import tqdm.notebook as tq
from nltk.translate.bleu_score import sentence_bleu

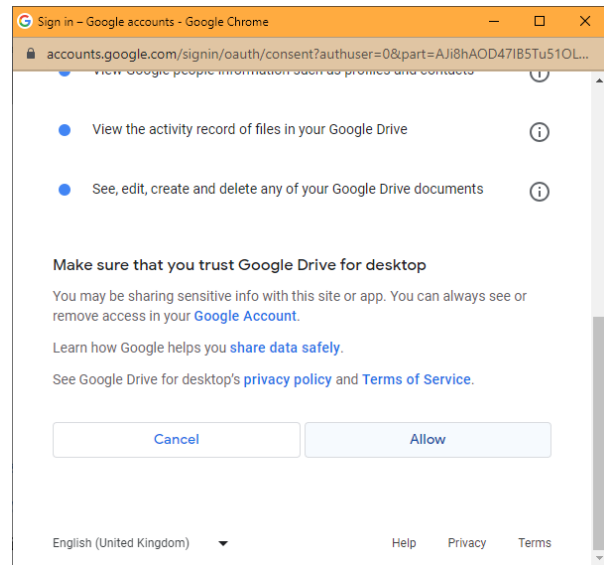
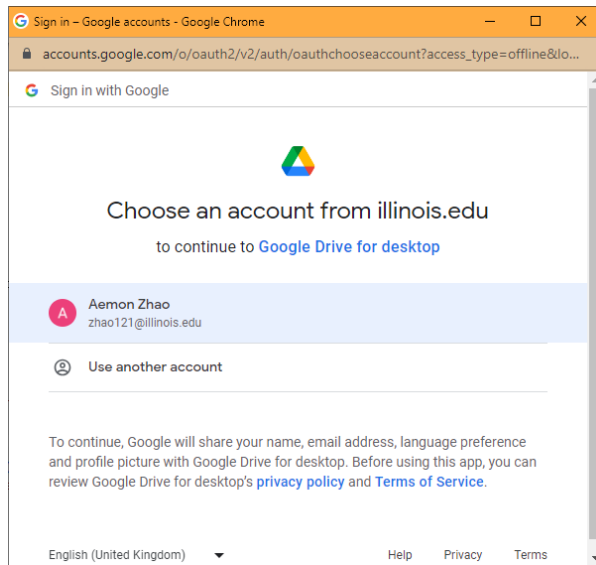
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f'GPU Available: {torch.cuda.is_available()}')
```

```
[ ] tokenizer = BertTokenizer.from_pretrained("fnlp/bart-base-chinese")
model = BartForConditionalGeneration.from_pretrained("fnlp/bart-base-chinese")
model = model.to(device)
```

Load the Dataset for Training your Model

```
[ ] # training_index = '300'
training_index = '900'
```

Executing (10s) <cell line: 2> > .mount() > .mount() > blocking_request() > read_reply_from_input()



2.1.5. Follow the instructions and tips given in the notebook file and from Step 2.2 of this guideline to run the cells and finish the tasks assigned in Section 3 of this guideline.

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/My Drive/Colab Notebooks/Lab2

Mounted at /content/drive
/content/drive/My Drive/Colab Notebooks/Lab2
```

Install Necessary Libraries and Import a Pre-trained Model

```
[ ] | pip install transformers

[ ] | import os
import shutil
import numpy as np
import torch
from transformers import BertTokenizer, BartForConditionalGeneration, get_linear_schedule_with_warmup
from torch.optim import AdamW
from torch.utils.data import Dataset, DataLoader
import tqdm.notebook as tq
from nltk.translate.bleu_score import sentence_bleu

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f'GPU Available: {torch.cuda.is_available()}')

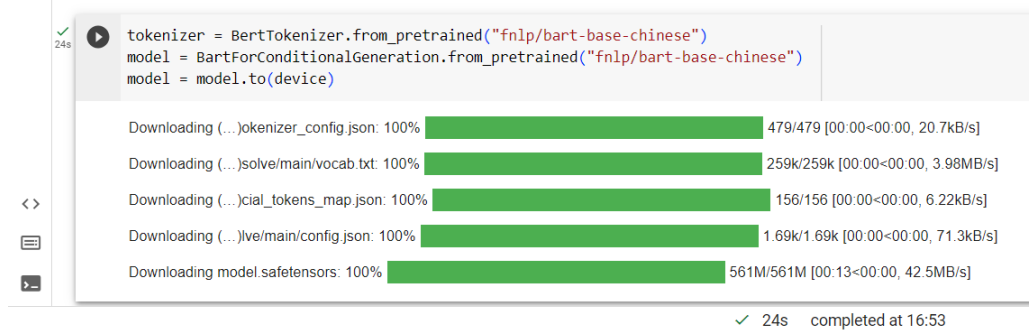
[ ] | tokenizer = BertTokenizer.from_pretrained("fnlp/bart-base-chinese")
model = BartForConditionalGeneration.from_pretrained("fnlp/bart-base-chinese")
model = model.to(device)
```

256 completed at 15:00

2.2. Training Processes and Tasks

2.2.1. Install Necessary Libraries and Import a Pre-trained Model

- Run the cells one by one.
- After running the last cell, you should see the following display.



```
tokenizer = BertTokenizer.from_pretrained("fnlp/bart-base-chinese")
model = BartForConditionalGeneration.from_pretrained("fnlp/bart-base-chinese")
model = model.to(device)
```

Downloading (...)okenizer_config.json: 100% 479/479 [00:00<00:00, 20.7kB/s]

Downloading (...)solve/main/vocab.txt: 100% 259k/259k [00:00<00:00, 3.98MB/s]

Downloading (...)cial_tokens_map.json: 100% 156/156 [00:00<00:00, 6.22kB/s]

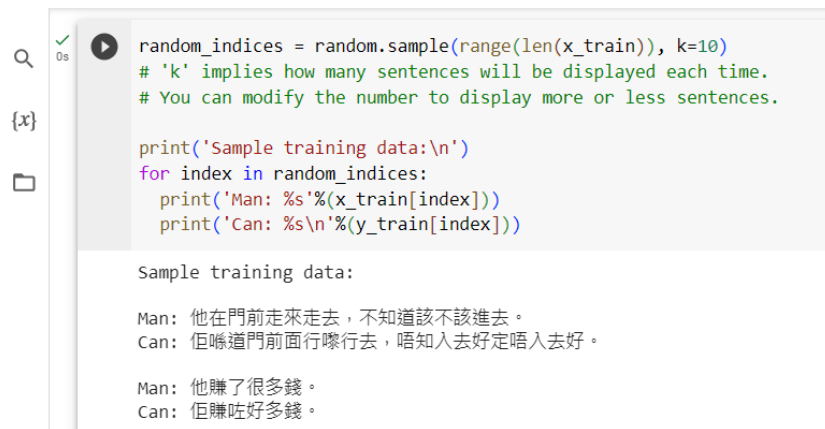
Downloading (...)lve/main/config.json: 100% 1.69k/1.69k [00:00<00:00, 71.3kB/s]

Downloading model.safetensors: 100% 561M/561M [00:13<00:00, 42.5MB/s]

✓ 24s completed at 16:53

2.2.2. Load the Dataset for Training your Model

- Run the cells one by one.
- You should see some sample Mandarin and corresponding Cantonese sentences in the dataset.



```
random_indices = random.sample(range(len(x_train)), k=10)
# 'k' implies how many sentences will be displayed each time.
# You can modify the number to display more or less sentences.

print('Sample training data:\n')
for index in random_indices:
    print('Man: %s'%(x_train[index]))
    print('Can: %s\n'%(y_train[index]))
```

Sample training data:

Man: 他在門前走來走去，不知道該不該進去。
Can: 佢喺道門前面行嚟行去，唔知入去好定唔入去好。

Man: 他賺了很多錢。
Can: 佢賺咗好多錢。

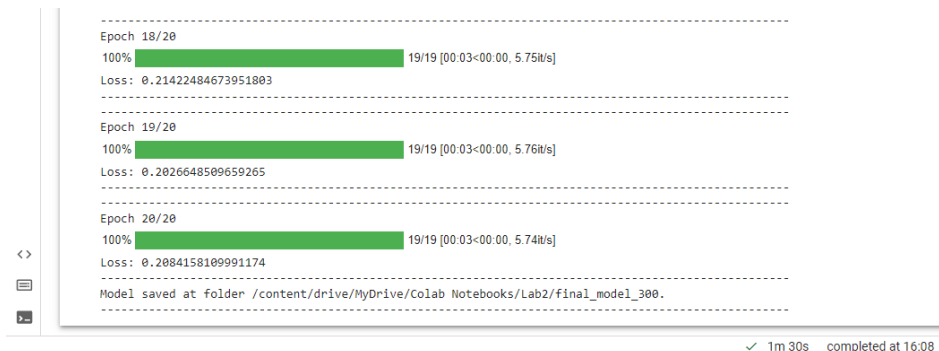
2.2.3. Tokenisation

- A sentence can be tokenised to a series of numbers and recovered from them using a tokeniser. Those processes are called "encoding" and "decoding". The below cell displays the encoded numbers and decoded results of a random sentence from the training dataset. You can also repeatedly run the below cell to see the demonstration in different sentences or sentences specified by yourself.

- Example:
 - “我認為大家都應該學另外一種語言。” (“I think we should all learn another language.”)
 - Encoded = [101, 9970, 20173, 13671, 7723, 8362, 21784, 9899, 20146, 8302, 6393, 7706, 4896, 16056, 20190, 20037, 3566, 102, 0, 0, 0, 0, 0, 0]
- **Task 1:** Obtain the encoded tokens of “然後我很快就睡著了。” (“After that, I soon fell asleep.”)

2.2.4. Fine-tuning

- Run the cells one by one.
- The “loss” should gradually decrease during training.
- After fine-tuning is done, the final model will be saved under the directory “/Colab Notebooks/Lab2/final_model_300”. The “300” here indicates that 300 pairs of Mandarin and Cantonese sentences are used to fine-tune the model.



2.2.5. Evaluation

- Given a test sentence: “她是一個非常美的女人。” (“She is a very beautiful woman.” in Mandarin), the programme could compare the output of the pre-trained model and the fine-tuned model and get the BLEU (Bi-Lingual Evaluation Understudy) score for the result with the reference translation, “佢係一個好靚嘅女人。” (“She is a very beautiful woman.” in Cantonese)
- A BLEU score should be a value between 0 and 1, with the value closer to 1 representing more similar texts between the input and the output, i.e., a better translation.
- **Task 2:** Evaluate the performance of the model before and after fine-tuning

- Translate the Mandarin sentence “她是一個非常美的女人。” using the two models and **report the BLEU scores of both translations.**

After fine-tuning:

```
✓ [15] translation = translate_sentence(test_sentence, final_model,
                                     tokenizer, device, MAX_LENGTH)
      translation = ' '.join(list(translation)).split()
      print("Translation:", ' '.join(translation))

      score = sentence_bleu([list(reference)], translation,
                             smoothing_function=SmoothingFunction().method4)
      print("BLEU score:", score)
```

Again, you can select and copy from the above sentences to [Bing Microsoft Tran](#) English **for reference only** if you wish.

- Use the same test set, “test_100.txt”, to evaluate and compare the pre-trained model and “final_model_300” by checking the average BLEU scores for each case. **Report the two BLEU scores for the pre-trained model and the model “final_model_300”.**

Computing the average BLEU score of the fine-tuned model on the test set:

```
[ ] bleu_score_list = []

with open("/content/drive/MyDrive/Colab Notebooks/Lab2/data/test_100.txt",
          'r') as f:
    for line in f:
        line = line.strip().split('\t')
        test_sentence = line[0]
        reference = line[1]

        translation = translate_sentence(test_sentence, final_model,
                                         tokenizer, device, MAX_LENGTH)
        translation = ' '.join(list(translation)).split()

        score = sentence_bleu([list(reference)], translation,
                               smoothing_function=SmoothingFunction().method4)
        bleu_score_list.append(score)

print("Average BLEU score of {0} sentences: {1}".\
      format(len(bleu_score_list), np.mean(bleu_score_list)))
```

- **Task 3: Evaluate the performance of the fine-tuned “final_model_300” on sentences with different lengths.**

- Sentence Length (SL): number of characters of the sentence. E.g., the length of “然後我很快就睡著了。” (“After that, I soon fell asleep.”) is 10: “然”, “後”, “我”, “很”, “快”, “就”, “睡”, “著”, “了”, “。”.

- Refer to another two test sets, each containing 20 sentences.

sentences_8-10.txt: SL ranges from 8 to 10.

sentences_20-30.txt: SL ranges from 20 to 30

Computing the average BLEU score of the fine-tuned model on the other sets:

```
bleu_score_list = []

with open("/content/drive/MyDrive/Colab Notebooks/Lab2/data/\
sentences_8-10.txt", 'r') as f:
    for line in f:
        line = line.strip().split('\t')
        test_sentence = line[0]
        reference = line[1]

        translation = translate_sentence(test_sentence, final_model,
                                         tokenizer, device, MAX_LENGTH)
        translation = ' '.join(list(translation)).split()

        score = sentence_bleu([list(reference)], translation,
                               smoothing_function=SmoothingFunction().method4)
        bleu_score_list.append(score)

print("Average BLEU score of {} sentences: {}".format(len(bleu_score_list), np.mean(bleu_score_list)))
```

You can modify the above-highlighted part to set the test set used.

- **Report the two average BLEU scores on the two test sets** and see if SL affects the model’s performance.
- **Task 4: Evaluate the performance of models trained with different amounts of data.**
 - Train another model with 900 sentences, “final_model_900”
 - Obtain the output of the newly fine-tuned model using the same example test sentences and **report the BLEU score**.
 - Use the same test set (test_100.txt) to evaluate the model and **report the average BLEU score**.
 - **Compare the BLEU scores with the ones that you obtained in Task 2 using “final_model_300”.** Observe if the two models have different performance. If yes, **try to explain why**.

- **Task 5 (Optional)**

Train a NMT model for *can2man*.

The NMT model in the demo is used to convert Mandarin to Cantonese. With the same model structure and parameters, train a model used to convert Cantonese to Mandarin and report the performance of the *can2man* model.